

Es. 1 - Programmazione concorrente

Si consideri un programma che deve risolvere un problema complesso. A questo scopo, il programma suddivide il problema in un certo numero di sottoproblemi indipendenti, che affida ad altrettanti thread slave, dopo di che fa sleep.

Quando un thread slave trova la soluzione, ne informa il main attraverso un interrupt (NB: usiamo interrupt per evitare che il main debba fare polling sugli slave).

Quando il main riceve l'interrupt, esce dalla sleep e deduce che un thread ha trovato la soluzione: a questo punto manda un interrupt ai thread slave, che ormai non servono più, affinché terminino l'esecuzione. Quando i thread slave ricevono l'interrupt, lasciano quel che stanno facendo e terminano.

Si noti quindi che gli interrupt hanno un significato diverso:

- quelli da slave a main indicano che si è trovata la soluzione
- quelli da main a slave indicano la richiesta di terminare il thread nello stato in cui si trova, senza più proseguire nella ricerca della soluzione.

Il codice dato non comprende la sincronizzazione: si chiede pertanto di modificarlo in modo che il programma si comporti come descritto sopra, evitando i problemi tipici della programmazione concorrente (corse critiche, deadlock, ecc.). NB: eventuali soluzioni che non utilizzano gli interrupt saranno considerate nulle.

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio 2. Programmazione distribuita (con uso di socket)

Si vuole realizzare un sistema distribuito che implementa delle funzionalità simili a quelle descritte nell'esercizio di programmazione concorrente.

Il server gestisce il problema complesso. I client sono i solutori: quando un client stabilisce la connessione col server riceve un sottoproblema da risolvere. NB: i solutori sono un numero qualunque: ogni solutore che si collega al server riceve un sottoproblema.

Quando il server accetta una connessione, crea uno slave che gestirà le comunicazioni col client.

Master e slave condividono un oggetto soluzione, in cui lo slave scriverà la soluzione ricevuta dal client.

Come nell'esercizio di programmazione concorrente, si vuole che, quando un client ha comunicato la soluzione, il server comunichi ai vari client che non è più necessario proseguire nella ricerca della soluzione, e che quindi possono terminare. NB: trattandosi di un sistema distribuito, la comunicazione tra server e client avviene mediante letture e scritture su socket anziché via interrupt.

È accettabile che ci sia un certo ritardo

- tra il momento in cui la soluzione viene trovata da un client e il momento in cui viene comunicata al corrispondente slave
- tra il momento in cui la soluzione viene ricevuta da uno slave e il momento in cui il server master se ne rende conto
- tra il momento in cui il server ha notizia della soluzione e il momento in cui tutti client terminano.

Suggerimento: è accettabile che i client eseguano iterativamente la seguente sequenza di operazioni:

- 1) elaborazione (durante la quale non si comunica col server)
- 2) comunicazione con il server slave: questa fase deve avere durata limitata e prevedibile, per non sottrarre troppo tempo all'elaborazione.

Si chiede di realizzare il programma descritto, eventualmente riusando parte del codice dell'applicazione concorrente locale, mediante l'uso di socket.

Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio 3. Programmazione distribuita (con uso di RMI)

Si vuole realizzare un sistema distribuito che implementa delle funzionalità simili a quelle descritte nell'esercizio di programmazione concorrente.

Il server gestisce il problema complesso. I client sono i solutori: quando un client stabilisce la connessione col server riceve un sottoproblema da risolvere. NB: i solutori sono un numero qualunque: ogni solutore che si collega al server riceve un sottoproblema.

Come nell'esercizio di programmazione concorrente si vuole che, quando un client ha comunicato la soluzione, il server comunichi ai vari client che non è più necessario proseguire nella ricerca della soluzione, e che quindi possono terminare. NB: trattandosi di un sistema RMI, la comunicazione tra server e client avviene mediante chiamate di metodi anziché via interrupt.

È accettabile che ci sia un certo ritardo (purché piccolo e prevedibile) tra le varie fasi e operazioni svolte nel sistema: ad es., tra il momento in cui il server ha notizia della soluzione e il momento in cui tutti client terminano.

Si chiede di realizzare il programma descritto, eventualmente riusando parte del codice dell'applicazione concorrente locale, mediante l'uso di RMI.

Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.